



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа №4
«Построение предсказательной модели»
по курсу «Моделирование»

Студент: Аригин М.А.

Группа: ИУ9-82Б

Преподаватель: Домрачева А.Б.

Москва 2026

1 Постановка задачи

Имеется датасет `diabetes.csv` из 768 наблюдений. В качестве зависимой переменной выбрана `Glucose` (уровень глюкозы). Контрольный признак `Outcome` не используется при построении модели. По условию также исключается признак `Insulin`. Таким образом, после исключений остаются 6 факторов:

$$X = (\text{Pregnancies}, \text{BloodPressure}, \text{SkinThickness}, \text{BMI}, \\ \text{DiabetesPedigreeFunction}, \text{Age}).$$

Цель работы — построить статистическую вычислительную модель, позволяющую по значениям факторов оценивать уровень глюкозы, а также (при необходимости) сократить число факторов, обосновав выбор.

Входные данные:

- файл `diabetes.csv` (https://disk.yandex.ru/i/M35XVBrxn09_iQ);
- $Y = \text{Glucose}$;
- X — перечисленные 6 признаков.

Описание признаков (семантика столбцов).

1. `Pregnancies` — количество беременностей (ед.).
2. `Glucose` — результат двухчасового перорального теста на толерантность к глюкозе (мг/дл).
3. `BloodPressure` — диастолическое артериальное давление (мм рт. ст.).
4. `SkinThickness` — толщина кожной складки трицепса (мм).
5. `Insulin` — результат двухчасового теста на содержание инсулина в сыворотке крови (мкЕд/мл).
6. `BMI` — индекс массы тела: вес (кг) / рост² (м²).
7. `DiabetesPedigreeFunction` — показатель наследственной предрасположенности (безразмерный).

8. Age — возраст пациента (полных лет).
9. Outcome — 1, если пациент отправлен на дополнительное обследование.

Выходные данные:

- выбранный набор факторов (оптимальная размерность p);
- оценённые параметры модели;
- результаты проверок гипотез и качества модели.

2 Построение математической модели

Тип модели. Задача относится к методам многомерного статистического анализа. Рассматривается стохастическая статистическая модель, поскольку входные данные представлены выборкой наблюдений (случайных величин/векторов), а искомая зависимость устанавливается по статистическим оценкам и проверкам гипотез. В терминах предсказательных моделей это означает, что строится отображение “объект $\rightarrow$ ответ”, а параметры выбираются по данным; при этом в регрессионной постановке учитывается случайная ошибка (шум) ε .

2.1 Формализация задачи в общем виде

Пусть задана выборка из n наблюдений $(x^{(k)}, y^{(k)})$, где $y^{(k)} \in \mathbb{R}$ — значение зависимой переменной (уровень глюкозы), а $x^{(k)} \in \mathbb{R}^p$ — вектор факторов. Требуется построить модель вида

$$\hat{y} = f(x; \theta),$$

где θ — вектор неизвестных параметров, оцененных по выборке.

Общий вид предсказательной модели. В общем виде предсказательная модель задаётся отображением

$$a : X \times W \rightarrow Y,$$

где X — множество объектов (векторов признаков), W — множество параметров модели, Y — множество ответов. Так как ответ Y является вещественным числом (Glucose), задача относится к *регрессии* (а не к классификации). Линейная модель регрессии (линейная по параметрам) в общем виде может быть записана как

$$a(x, w) = \langle x, w \rangle,$$

а в частном виде, с выделением свободного члена, — как

$$a(x, \beta) = \beta_0 + \sum_{j=1}^p \beta_j x_j.$$

Связь с постановкой данной ЛР. В нашей задаче $y^{(k)} = \text{Glucose}^{(k)}$, а вектор факторов формируется после исключения Outcome и Insulin. Далее допускается уменьшить p за счёт отбора факторов по результатам рангового анализа и устойчивости регрессионной оценки.

2.2 Ранговая гипотеза (Спирмен)

Так как коэффициент Спирмена измеряет тесноту монотонной связи, выдвигается гипотеза о наличии монотонной зависимости между Y и частью факторов X_j . Для каждого фактора проверяется гипотеза:

$$H_0 : \rho_S(Y, X_j) = 0, \quad H_1 : \rho_S(Y, X_j) \neq 0,$$

где ρ_S вычисляется по формуле Спирмена

$$\rho_S = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)},$$

а d_i — разность рангов в паре наблюдений. с использованием приближённой t -статистики (обозначим $K = n - 2$ — число степеней свободы):

$$t = \rho_S \sqrt{\frac{K}{1 - \rho_S^2}}, \quad \text{df} = K, \quad \alpha = 0.05.$$

Эквивалентно можно сравнивать $|\rho_S|$ с критической точкой

$$\rho_{crit} = \frac{t_{crit}}{\sqrt{t_{crit}^2 + K}}.$$

2.3 Итоговая модель

По результатам предварительного анализа степенные/экспоненциальные функции связи для отдельных пар признаков не дают устойчивой адекватности (в частности, по критерию Колмогорова–Смирнова) — подробно процедура подбора вида связи описана в разделе 3.5. Поэтому в качестве *отправной точки* (базы для сравнения) берётся простое интерпретируемое приближение — множественная линейная регрессия:

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon, \quad E[\varepsilon] = 0.$$

Оценка параметров выполняется методом моментов (нормальные уравнения):

$$(Z^T Z) \hat{\beta} = Z^T y, \quad \hat{\beta} = (Z^T Z)^{-1} Z^T y.$$

Для устойчивости масштабов в вычислениях факторы приводятся к сопоставимому масштабу по рабочей выборке. Так как реальные данные содержат выбросы и «аномальные» значения, используется робастная нормализация на основе медианы и межквартильного размаха (IQR). Согласно методическим замечаниям, выбросы не следует безосновательно исключать из выборки; вместо этого применяется *цензурирование* (ограничение области определения/допустимых значений) и робастные оценки масштаба.

Переход к нелинейной модели. Если линейная регрессия окажется неадекватной на контрольной выборке (см. раздел 5.6), то нелинейную зависимость сводят к линейной заменой переменных: вводят преобразования признаков $g_j(x_j)$ и при необходимости отклика $h(y)$, после чего применяют тот же многомерный МНК уже к преобразованным признакам:

$$h(\hat{y}) = \beta_0 + \sum_{j=1}^p \beta_j g_j(x_j), \quad \hat{y} = h^{-1} \left(\beta_0 + \sum_{j=1}^p \beta_j g_j(x_j) \right).$$

Подробная процедура подбора g_j , h и список рассмотренных вариантов модели (включая аддитивные нелинейности, парные интеракции, расширение до всех 6 признаков) описаны ниже в разделах 3.5–5.7.

2.4 Схема модели

Для наглядности последовательность вычислительных этапов представлена на схеме (рис. 1).

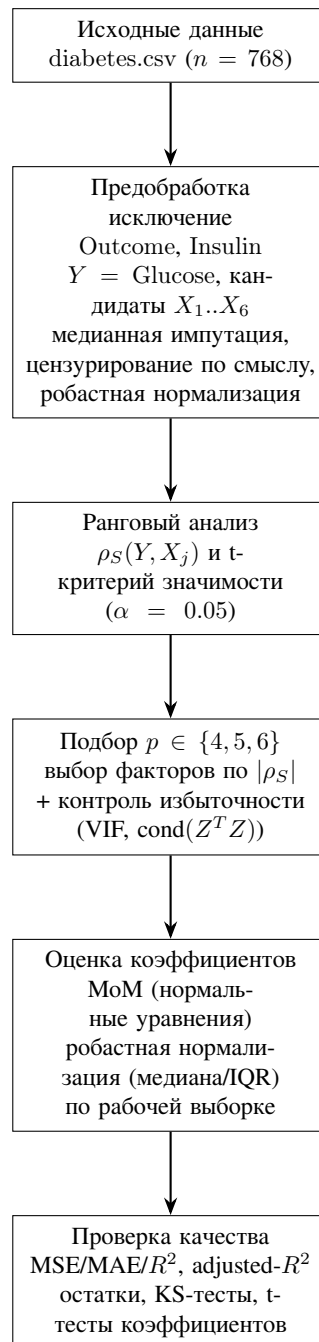


Рисунок 1 Схема вычислительной модели регрессии уровня глюкозы.

3 Решение вычислительных задач

3.1 Разделение выборки и робастная предобработка

Для оценки качества модели исходная выборка разделяется на рабочую и контрольную части в пропорции 50/50 (случайная перестановка с фиксированным $\text{seed}=42$). Все параметры предобработки вычисляются только по рабочей части и затем применяются к контрольной (во избежание утечки информации).

Удаление недопустимых значений отклика. В исходных данных 5 записей имеют $\text{Glucose} = 0$, что является недопустимым значением для этого признака и трактуется как явный пропуск измерения. Поскольку OLS чувствителен к таким аномалиям отклика, эти строки исключаются *до* разделения; рабочая объёмом $n_r = 381$ и контрольная объёмом $n_k = 382$ строятся из оставшихся $n = 763$.

Проверка корректности разделения. Так как данные не упорядочены, важно убедиться, что рабочая и контрольная выборки сопоставимы по распределению отклика. Для этого применяется двухвыборочный критерий Колмогорова–Смирнова к $Y_{\text{рабоч}}$ и $Y_{\text{контр}}$:

$$\text{KS}(Y_{\text{рабоч}}, Y_{\text{контр}}) : D = 0.104, p\text{-value} = 0.028.$$

Эффект малый ($D \approx 0.1$), но на уровне $\alpha = 0.05$ однородность распределения формально *не подтверждается*. Это побочный эффект удаления 5 выбросов отклика: при случайной перестановке отброшенных строк больше попало в одну часть. Однако для сравнения моделей это не критично: обе модели (линейная и нелинейная) обучаются на одной и той же рабочей и проверяются на одной и той же контрольной выборке, поэтому относительный выигрыш одной над другой не смещён. Для воспроизводимости фиксируется seed , что гарантирует одинаковое разделение при повторных запусках.

Медианная обработка. Для факторов BloodPressure , SkinThickness , BMI значение 0 трактуется как пропуск. Пропуски заполняются медианой по рабочей выборке.

Выбросы и цензурирование. Наличие аномалий оценивается по квартилям: сравниваются расстояния $d_L = Q_2 - Q_1$ и $d_R = Q_3 - Q_2$. Существенное различие

d_L и d_R указывает на асимметрию и возможное влияние выбросов на средние оценки. При этом наблюдения не исключаются «вручную»; вместо этого:

- при наличии априорной области допустимых значений используется цензурирование (ограничение значений областью). В рамках данной работы цензурирование используется как *смысловое ограничение области*: для BloodPressure, SkinThickness, BMI принимается, что значения должны быть строго положительными, поэтому нулевые значения трактуются как отсутствие измерения и заменяются медианой по рабочей выборке (что эквивалентно цензурированию слева на границе 0 с последующей робастной оценкой); для Pregnancies допускается 0 (валидное значение), для DiabetesPedigreeFunction допускается 0 (показатель может быть равен 0).
- в расчётах применяется робастная нормализация $(x - \text{median})/IQR$, уменьшающая влияние экстремальных значений.

Вывод по данным. В ноутбуке отдельно построена таблица квартилей и величин d_L , d_R , IQR для каждого признака. Если для признака наблюдается заметная асимметрия (d_R/d_L сильно отличается от 1), то признак потенциально содержит аномальные наблюдения, влияющие на средние оценки. Поэтому далее используется медианная обработка пропусков и робастная нормализация; исключение наблюдений не выполняется.

3.2 Расчёт коэффициентов Спирмена и проверка значимости

Вычисляются значения $\rho_S(Y, X_j)$ и t-статистика, затем принимается решение о значимости связи при $\alpha = 0.05$. Таблица результатов экспортируется из ноутбука.

3.3 Сокращение размерности

Рассматривается перебор числа факторов $p \in \{4, 5, 6\}$. Для каждого p решается задача отбора подмножества факторов из 6 кандидатов.

Идея отбора. Так как Спирмен измеряет монотонную связь, в качестве первичного ранжирования факторов используется величина $|\rho_S(Y, X_j)|$: чем она больше, тем сильнее статистическая зависимость фактора от отклика. Однако простое «взять top- p » может привести к избыточности факторов (мультиколлинеарности), что ухудшает устойчивость оценки $\hat{\beta}$.

Контроль избыточности. Для выбранного набора факторов дополнительно вычисляются:

- число обусловленности $\text{cond}(Z^T Z)$;
- максимальный VIF (variance inflation factor), рассчитанный по корреляционной матрице факторов.

Малый VIF (близкий к 1) свидетельствует об отсутствии сильной линейной зависимости между факторами.

Критерий оптимальности. В качестве основного критерия сравнения моделей по p берутся метрики на контрольной выборке (RMSE/MSE/MAE/ R^2) и adjusted- R^2 (при этом RMSE вычисляется вручную как $\text{RMSE} = \sqrt{\text{MSE}}$). Итоговый выбор делается из принципа простоты модели: выбирается минимальное p , для которого качество практически не хуже лучшего варианта, при приемлемой устойчивости (VIF, cond).

3.4 Оценивание коэффициентов

По выбранному набору факторов оцениваются $\hat{\beta}$ методом моментов и вычисляются прогнозы на контрольной выборке.

3.5 Подход к построению нелинейной модели через линеаризацию

Базовая линейная модель $\hat{y} = \beta_0 + \sum_j \beta_j x_j^*$ показывает на контрольной выборке $R^2 \approx 0.088$, $\text{MAE}_{\text{контр}} \approx 23.50$, $\text{RMSE}_{\text{контр}} \approx 30.13$. На графике y vs $\hat{y}$ виден характерный эффект «стягивания к среднему» (см. рис. 2): при больших y прогноз систематически занижен, при малых — завышен. Это свидетельствует,

что связь X и Y *не линейна* в исходных координатах, и линейная регрессия в таком виде **неадекватна**.

Идея. Сведём нелинейную модель к линейной заменой переменных: введём преобразования g_j для признаков и h для отклика так, чтобы регрессия стала линейной по параметрам:

$$h(\hat{y}) = \beta_0 + \sum_{j=1}^p \beta_j g_j(x_j), \quad \hat{y} = h^{-1}\left(\beta_0 + \sum_{j=1}^p \beta_j g_j(x_j)\right). \quad (1)$$

Это позволяет применить уже отработанный многомерный МНК к *нелинейным* признакам $g_j(x_j)$ и сохранить интерпретируемость и устойчивость линейной оценки.

Кандидаты на преобразование. Для признаков: $g_j(x) \in \{x, \sqrt{x}, \ln x, x^2, 1/x\}$ (допустимы только при корректной области: $x > 0$ для $\ln$ и $1/x$, $x \geq 0$ для $\sqrt{}$). Для отклика: $h(y) \in \{y, \ln y, \sqrt{y}\}$. Дополнительно рассмотрена расширенная нелинейность с парными произведениями $g_j(x_j) \cdot g_k(x_k)$ (умножение признаков — ещё одна типичная нелинейность курса; полученная модель остаётся линейной по β).

Как выбрать g_j . Замечание: коэффициент Спирмена ρ_S *инвариантен* относительно монотонных преобразований, поэтому он одинаков для $\{x, \sqrt{x}, \ln x\}$ и не различает их. Для подбора *линеаризующего* преобразования используется линейный коэффициент корреляции Пирсона $r(g_j(x_j), h(y))$: он чувствителен к виду нелинейности и максимизируется при удачном «выпрямлении». В качестве вспомогательной проверки (по правилу: «если оценка β близка к 2 — берём $\sqrt{x}$, при экспоненциальном росте — логарифм») для каждого признака отдельно оценивается показатель $\hat{\beta}$ в одномерной степенной модели $y \approx a \cdot x^\beta$ через лог-линейный МНК на положительной подвыборке ($\ln y = \ln a + \beta \ln x$).

Алгоритм построения нелинейной модели.

1. Все вычисления параметров (Pearson, медианы, $\hat{\beta}$ степенной модели) выполняются *только по рабочей выборке*.
2. Для каждого допустимого $h(y)$ и каждого признака x_j перебираются допустимые g_j , выбирается g_j с максимальным $|r(g_j(x_j), h(y))|$.

3. Для каждого h строится линейный МНК (методом моментов) на $Z = [1, g_1(x_1), \dots, g_p(x_p)]$; прогноз на контрольной: $\hat{y}_{te} = h^{-1}(Z_{te}\hat{\beta})$.
4. Среди всех кандидатов выбирается модель с минимальной $\text{MAE}_{\text{контр}}$ — критерий, измеряющий обобщающую способность, а не подгонку.

Дополнительно проверены ещё три принципиальные конфигурации нелинейной модели:

- **(M3) Все 6 кандидатных факторов** с подобранными по Pearson g_j — проверка, помогает ли увеличение размерности нелинейной части.
- **(M4) Аддитивная нелинейность** — к 4 базовым x_j для каждого признака добавляется второй базис $g_j(x_j)$, если non-identity-преобразование значимо сильнее по $|r|$. Получаем расширенный базис, где каждый признак вносит *и* линейный, *и* нелинейный вклад.
- **(M5) Одиночная парная интеракция** $g_j(x_j) \cdot g_k(x_k)$, выбранная по $\max |r(g_j g_k, y)|$ на рабочей выборке. Это «умножение признаков», на которое указывает методическое замечание; вводится экономно (одна интеракция вместо всех), чтобы избежать переобучения.

Все варианты обучаются на одной и той же рабочей выборке, проверяются на одной и той же контрольной; победитель определяется минимумом $\text{MAE}_{\text{контр}}$. См. таблицу 5 и обсуждение в разделе 5.6.

Выбор метрик. Для сравнения линейной и линеаризованной нелинейной моделей применяются: **MAE** (обязательная), **RMSE**, R^2 . MAE менее чувствителен к выбросам, чем RMSE, и поэтому удобен для оценки практической точности модели. Все метрики вычисляются как на рабочей, так и на контрольной выборке; основной вывод об адекватности делается **по контрольной**.

4 Практическая реализация

Listing 1: Модель линейной регрессии

```
1 import math
2 import os
```

```

3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7
8  DIR = Path(".")
9  ALPHA = 0.05
10 RANDOM_SEED = 42
11
12
13 TEST_SIZE = 0.5
14
15 os.makedirs("tables", exist_ok=True)
16 os.makedirs("images", exist_ok=True)
17
18
19 _df = pd.read_csv("diabetes.csv")
20 Y_NAME = "Glucose"
21 DROP = {"Outcome", "Insulin"}
22 X_NAMES = [c for c in _df.columns if c not in DROP and c != Y_NAME]
23
24 Y = _df[Y_NAME].astype(float)
25 X = _df[X_NAMES].astype(float)
26
27 print("Y =", Y_NAME)
28 print("X columns (after dropping Outcome, Insulin):", X_NAMES)
29 print("n =", len(_df))
30
31 ZERO_AS_MISSING = {"BloodPressure", "SkinThickness", "BMI"}
32
33 zero_counts = (X == 0).sum().sort_values(ascending=False)
34 print("Zero counts in columns:")
35 display(pd.DataFrame({"zeros": zero_counts}))
36 print("ZERO_AS_MISSING used:", sorted(ZERO_AS_MISSING))
37
38
39 def quantile_outlier_summary(df: pd.DataFrame, ratio_lo=0.5, ratio_hi=2.0):
40     rows = []
41     for c in df.columns:
42         s = df[c].astype(float).copy()
43
44         if c in ZERO_AS_MISSING:
45             s = s.replace(0, np.nan)
46             s = s.dropna()
47             q1 = float(np.quantile(s, 0.25))
48             q2 = float(np.quantile(s, 0.50))
49             q3 = float(np.quantile(s, 0.75))

```

```

50     dL = q2 - q1
51     dR = q3 - q2
52     iqr = q3 - q1
53     ratio = float(dR / dL) if dL != 0 else float("inf")
54     flag = (ratio < ratio_lo) or (ratio > ratio_hi)
55     rows.append(
56         {
57             "feature": c,
58             "Q1": q1,
59             "Q2(median)": q2,
60             "Q3": q3,
61             "dL=Q2-Q1": dL,
62             "dR=Q3-Q2": dR,
63             "ratio=dR/dL": ratio,
64             "IQR": iqr,
65             "asymmetry_flag": flag,
66         }
67     )
68     return pd.DataFrame(rows).sort_values("asymmetry_flag", ascending=False)
69
70 outlier_tbl = quantile_outlier_summary(X)
71 display(outlier_tbl)
72
73 n_flag = int(outlier_tbl["asymmetry_flag"].sum())
74 print(f"Asymmetry flags: {n_flag} / {len(outlier_tbl)} features")
75
76 IQR_MAX = {}
77 if IQR_MAX:
78     outlier_tbl["IQR_max"] = outlier_tbl["feature"].map(IQR_MAX)
79     outlier_tbl["IQR_exceeds"] = outlier_tbl.apply(
80         lambda r: (r["IQR"] > r["IQR_max"]) if pd.notna(r["IQR_max"]) else False,
81         axis=1,
82     )
83     display(outlier_tbl[["feature", "IQR", "IQR_max", "IQR_exceeds"]])
84 else:
85     print("IQR_MAX not set.")
86
87
88 rho_y = X.corrwith(Y, method="spearman")
89 n = len(Y)
90 K = n - 2 # число степеней свободы
91 rho_vals = rho_y.to_numpy(float)
92
93 # t-статистика значимости
94 t_calc = rho_vals * np.sqrt(K / np.maximum(1e-12, 1.0 - rho_vals**2))
95
96 try:

```

```

97     from scipy.stats import t as student_t
98
99     t_crit = float(student_t.ppf(1 - ALPHA / 2, K))
100 except Exception:
101     t_crit = 1.959964
102
103     # Критическая точка по  $|\rho_S|$  через t_crit:
104     #  $\rho_{crit} = t_{crit} / \sqrt{t_{crit}^2 + K}$ 
105     rho_crit = float(t_crit / np.sqrt(t_crit**2 + K))
106
107 spearman_signif = pd.DataFrame(
108     {
109         "Xj": rho_y.index,
110         "rho_S": rho_vals,
111         "n": n,
112         "K": K,
113         "alpha": ALPHA,
114         "t_crit": t_crit,
115         "t_calc": t_calc,
116         "rho_crit": rho_crit,
117         "decision": [
118             "reject H0" if abs(r) >= rho_crit else "fail to reject H0" for r in rho_vals
119         ],
120     }
121 ).sort_values("rho_S", key=lambda s: s.abs(), ascending=False)
122
123 spearman_signif
124
125 spearman_signif.to_csv("tables/lab4_spearman_yx_significance.csv", index=False,
126     ↪ encoding="utf-8-sig")
127
128 try:
129     spearman_signif_round = spearman_signif.copy()
130     for c in ["rho_S", "t_calc", "t_crit", "rho_crit"]:
131         spearman_signif_round[c] = spearman_signif_round[c].astype(float).round(6)
132     spearman_signif_round = spearman_signif_round.rename(
133         columns={
134             "rho_S": "$\\rho_S$",
135             "t_crit": "$t_{crit}$",
136             "t_calc": "$t_{calc}$",
137             "rho_crit": "$\\rho_{crit}$",
138             "alpha": "$\\alpha$",
139             "K": "$K$",
140         }
141     )
142     spearman_signif_round.to_latex(
143         "tables/lab4_spearman_yx_significance.tex",
144         index=False,

```

```

143         escape=False,
144     )
145 except Exception as e:
146     print("LaTeX export failed:", repr(e))
147
148 print("Saved tables/lab4_spearman_yx_significance.csv and .tex")
149
150 def train_test_split_idx(n: int, test_size: float, seed: int):
151     rng = np.random.default_rng(seed)
152     idx = np.arange(n)
153     rng.shuffle(idx)
154     m = int(round(n * test_size))
155     test = np.sort(idx[:m])
156     train = np.sort(idx[m:])
157     return train, test
158
159 def median_impute_fit(Xm: np.ndarray):
160     med = np.nanmedian(Xm, axis=0)
161     return med
162
163 def median_impute_apply(Xm: np.ndarray, med: np.ndarray):
164     Xm2 = Xm.copy()
165     mask = ~np.isfinite(Xm2)
166     if np.any(mask):
167         Xm2[mask] = np.take(med, np.where(mask)[1])
168     return Xm2
169
170 def iqr_clip_fit(Xm: np.ndarray, k: float = 1.5):
171     q1 = np.quantile(Xm, 0.25, axis=0)
172     q3 = np.quantile(Xm, 0.75, axis=0)
173     iqr = q3 - q1
174     lo = q1 - k * iqr
175     hi = q3 + k * iqr
176     return lo, hi
177
178 def iqr_clip_apply(Xm: np.ndarray, lo: np.ndarray, hi: np.ndarray):
179     return np.clip(Xm, lo, hi)
180
181 def robust_scale_fit(Xm: np.ndarray):
182     center = np.median(Xm, axis=0)
183     q1 = np.quantile(Xm, 0.25, axis=0)
184     q3 = np.quantile(Xm, 0.75, axis=0)
185     scale = q3 - q1
186     scale = np.where(scale == 0, 1.0, scale)
187     return center, scale
188
189 def robust_scale_apply(Xm: np.ndarray, center: np.ndarray, scale: np.ndarray):

```

```

190     return (Xm - center) / scale
191
192 def mom_beta(Z: np.ndarray, y: np.ndarray, ridge: float = 1e-10):
193     A = Z.T @ Z + ridge * np.eye(Z.shape[1])
194     b = Z.T @ y
195     return np.linalg.solve(A, b)
196
197 def metrics(y: np.ndarray, yhat: np.ndarray):
198     y = np.asarray(y, float)
199     yhat = np.asarray(yhat, float)
200     mse = float(np.mean((y - yhat) ** 2))
201     rmse = float(np.sqrt(mse))
202     mae = float(np.mean(np.abs(y - yhat)))
203     ss_res = float(np.sum((y - yhat) ** 2))
204     ss_tot = float(np.sum((y - float(np.mean(y))) ** 2))
205     r2 = float(1.0 - ss_res / ss_tot) if ss_tot > 0 else float("nan")
206     return mse, rmse, mae, r2
207
208 def adjusted_r2(r2: float, n: int, p: int):
209     if not np.isfinite(r2) or n - p - 1 <= 0:
210         return float("nan")
211     return float(1.0 - (1.0 - r2) * (n - 1) / (n - p - 1))
212
213 def vif_from_corr(corr: pd.DataFrame):
214     R = corr.to_numpy(float)
215     R = R + 1e-12 * np.eye(R.shape[0])
216     inv = np.linalg.inv(R)
217     return pd.Series(np.diag(inv), index=corr.index)
218
219
220 def greedy_by_rho_and_redundancy(
221     ordered_feats: list[str], abs_rho_xx: pd.DataFrame, k: int, max_pair_abs_rho: float
222 ):
223     selected: list[str] = []
224     for f in ordered_feats:
225         if not selected:
226             selected.append(f)
227             if len(selected) >= k:
228                 break
229             continue
230         mx = float(abs_rho_xx.loc[f, selected].max())
231         if mx <= max_pair_abs_rho:
232             selected.append(f)
233             if len(selected) >= k:
234                 break
235     if len(selected) < k:
236         for f in ordered_feats:

```

```

237         if f not in selected:
238             selected.append(f)
239         if len(selected) >= k:
240             break
241     return selected
242
243
244 n = len(Y)
245 tr_idx, te_idx = train_test_split_idx(n, TEST_SIZE, RANDOM_SEED)
246 abs_rho_xx = X.corr(method="spearman").abs()
247 ordered = rho_y.abs().sort_values(ascending=False).index.tolist()
248
249 rows = []
250 solutions = {}
251
252 for p in [4, 5, 6]:
253     best = None
254     best_key = None
255     for thr in [0.40, 0.50, 0.60, 0.70, 0.80, 1.00]:
256         sel = greedy_by_rho_and_redundancy(ordered, abs_rho_xx, k=p, max_pair_abs_rho=thr)
257
258         Xm = X[sel].to_numpy(float)
259         Xtr, Xte = Xm[tr_idx], Xm[te_idx]
260         ytr, yte = Y.to_numpy(float)[tr_idx], Y.to_numpy(float)[te_idx]
261
262         ZERO_AS_MISSING = {"BloodPressure", "SkinThickness", "BMI"}
263         Xtr0 = Xtr.copy()
264         Xte0 = Xte.copy()
265         for j, name in enumerate(sel):
266             if name in ZERO_AS_MISSING:
267                 Xtr0[Xtr0[:, j] == 0, j] = np.nan
268                 Xte0[Xte0[:, j] == 0, j] = np.nan
269
270         med = median_impute_fit(Xtr0)
271         Xtr1 = median_impute_apply(Xtr0, med)
272         Xte1 = median_impute_apply(Xte0, med)
273
274         center, scale = robust_scale_fit(Xtr1)
275         Xtr_s = robust_scale_apply(Xtr1, center, scale)
276         Xte_s = robust_scale_apply(Xte1, center, scale)
277
278         Ztr = np.column_stack([np.ones(len(tr_idx)), Xtr_s])
279         Zte = np.column_stack([np.ones(len(te_idx)), Xte_s])
280
281         beta = mom_beta(Ztr, ytr)
282         yhat_tr = Ztr @ beta
283         yhat_te = Zte @ beta

```

```

284
285     mse_tr, rmse_tr, mae_tr, r2_tr = metrics(ytr, yhat_tr)
286     mse_te, rmse_te, mae_te, r2_te = metrics(yte, yhat_te)
287     adj_te = adjusted_r2(r2_te, n=len(te_idx), p=p)
288
289     cond = float(np.linalg.cond(Ztr.T @ Ztr))
290     vif = vif_from_corr(X[sel].corr().fillna(0))
291     vif_max = float(vif.max())
292
293     cand = {
294         "p": p,
295         "thr": thr,
296         "selected": sel,
297         "mse_test": mse_te,
298         "rmse_test": rmse_te,
299         "mae_test": mae_te,
300         "r2_test": r2_te,
301         "adj_r2_test": adj_te,
302         "cond_ZtZ": cond,
303         "vif_max": vif_max,
304     }
305
306     if best is None or cand["mse_test"] < best["mse_test"]:
307         best = cand
308         best_key = (p, thr)
309         solutions[best_key] = {
310             **cand,
311             "beta": beta,
312             "med": med,
313             "center": center,
314             "scale": scale,
315         }
316     rows.append(best)
317 sweep_df = pd.DataFrame(rows)
318
319 best_mse = float(sweep_df["mse_test"].min())
320 within = sweep_df[sweep_df["mse_test"] <= 1.02 * best_mse].sort_values(["p"])
321 final_row = within.iloc[0]
322 final_p = int(final_row["p"])
323 final_thr = float(final_row["thr"])
324 final_selected = list(final_row["selected"])
325
326 print("Sweep (best thr per p):")
327 display(sweep_df)
328 print("\nFinal choice by parsimony:")
329 print("p=", final_p, "thr=", final_thr)
330 print("selected:", final_selected)

```

```

331
332
333 sweep_df.to_csv("tables/lab4_feature_sweep_p4_6.csv", index=False, encoding="utf-8-sig")
334 try:
335     sweep_tex = sweep_df.copy()
336     sweep_tex = sweep_tex.rename(
337         columns={
338             "mse_test": "$\\mathrm{MSE}_{\\text{test}}$",
339             "rmse_test": "$\\mathrm{RMSE}_{\\text{test}}$",
340             "mae_test": "$\\mathrm{MAE}_{\\text{test}}$",
341             "r2_test": "$R^2_{\\text{test}}$",
342             "adj_r2_test": "$\\overline{R}^2_{\\text{test}}$",
343             "cond_ZtZ": "$\\mathrm{cond}(Z^T Z)$",
344             "vif_max": "$\\max\\,\\,\\,\\mathrm{VIF}$",
345         }
346     )
347     sweep_tex.to_latex(
348         "tables/lab4_feature_sweep_p4_6.tex",
349         index=False,
350         escape=False,
351     )
352 except Exception:
353     pass
354
355 final_choice = pd.DataFrame({"selected": final_selected})
356 final_choice.to_csv("tables/lab4_final_selected_features.csv", index=False, encoding="utf-8-sig")
357 print("Saved tables/lab4_feature_sweep_p4_6.csv and lab4_final_selected_features.csv")
358
359 sol = solutions[(final_p, final_thr)]
360
361 Xm = X[final_selected].to_numpy(float)
362 Xtr, Xte = Xm[tr_idx], Xm[te_idx]
363 ytr, yte = Y.to_numpy(float)[tr_idx], Y.to_numpy(float)[te_idx]
364
365 ZERO_AS_MISSING = {"BloodPressure", "SkinThickness", "BMI"}
366 Xtr0 = Xtr.copy()
367 Xte0 = Xte.copy()
368 for j, name in enumerate(final_selected):
369     if name in ZERO_AS_MISSING:
370         Xtr0[Xtr0[:, j] == 0, j] = np.nan
371         Xte0[Xte0[:, j] == 0, j] = np.nan
372
373 med = sol["med"]
374 Xtr1 = median_impute_apply(Xtr0, med)
375 Xte1 = median_impute_apply(Xte0, med)
376
377 center, scale = sol["center"], sol["scale"]

```

```

378 Xtr_s = robust_scale_apply(Xtr1, center, scale)
379 Xte_s = robust_scale_apply(Xte1, center, scale)
380
381 Ztr = np.column_stack([np.ones(len(tr_idx)), Xtr_s])
382 Zte = np.column_stack([np.ones(len(te_idx)), Xte_s])
383
384 beta = sol["beta"]
385 yhat_tr = Ztr @ beta
386 yhat_te = Zte @ beta
387
388 mse_tr, rmse_tr, mae_tr, r2_tr = metrics(ytr, yhat_tr)
389 mse_te, rmse_te, mae_te, r2_te = metrics(yte, yhat_te)
390 adj_te = adjusted_r2(r2_te, n=len(te_idx), p=final_p)
391
392 coef_table = pd.DataFrame(
393     {
394         "term": ["Intercept"] + final_selected,
395         "beta": beta,
396     }
397 )
398
399 display(coef_table)
400
401 print("train: MSE=%.6f RMSE=%.6f MAE=%.6f R2=%.6f" % (mse_tr, rmse_tr, mae_tr, r2_tr))
402 print("test : MSE=%.6f RMSE=%.6f MAE=%.6f R2=%.6f adjR2=%.6f" % (mse_te, rmse_te,
    ↪   mae_te, r2_te, adj_te))
403
404 coef_table.to_csv("tables/lab4_final_regression_coeffs.csv", index=False, encoding="utf-8-sig")
405 try:
406     coef_table.to_latex("tables/lab4_final_regression_coeffs.tex", index=False)
407 except Exception:
408     pass
409
410 print("Saved tables/lab4_final_regression_coeffs.csv")
411
412 terms = ["Intercept"] + final_selected
413 beta_map = dict(zip(terms, beta))
414
415 rhs = " + ".join([f"({beta_map[name]:.6f}) · {name}" for name in final_selected])
416 model_str = f"yhat = ({beta_map['Intercept']:.6f}) + " + rhs
417 print("\nLinear model form (robust-scaled predictors):")
418 print(model_str)
419
420 import matplotlib.pyplot as plt
421 import numpy as np
422
423 fig, ax = plt.subplots(1, 2, figsize=(12, 5))

```

```

424 ax[0].scatter(yte, yhat_te, s=18, alpha=0.7)
425 mn = float(min(yte.min(), yhat_te.min()))
426 mx = float(max(yte.max(), yhat_te.max()))
427 ax[0].plot([mn, mx], [mn, mx], color="black", linewidth=1, label="y = yhat")
428 ax[0].set_xlabel("y (control)")
429 ax[0].set_ylabel("yhat (control)")
430 ax[0].set_title("Prediction: y vs yhat")
431 ax[0].grid(True, alpha=0.3)
432 ax[0].legend()
433
434
435 ord_idx = np.arange(len(yte))
436 ax[1].plot(ord_idx, yte, label="y (control)")
437 ax[1].plot(ord_idx, yhat_te, label="yhat (control)")
438 ax[1].set_xlabel("observation index")
439 ax[1].set_title("y and yhat on control set")
440 ax[1].grid(True, alpha=0.3)
441 ax[1].legend()
442
443 fig.tight_layout()
444 fig.savefig("images/lab4_prediction_vs_true.png", dpi=200)
445 plt.show()
446 print("Saved images/lab4_prediction_vs_true.png")
447
448 resid_tr = ytr - yhat_tr
449 resid_te = yte - yhat_te
450
451 resid_summary = pd.DataFrame(
452     {
453         "split": ["train", "test"],
454         "mean": [float(np.mean(resid_tr)), float(np.mean(resid_te))],
455         "std": [float(np.std(resid_tr, ddof=0)), float(np.std(resid_te, ddof=0))],
456     }
457 )
458 resid_summary
459
460 def ks_2sample_pvalue(x: np.ndarray, y: np.ndarray):
461     x = np.asarray(x, float)
462     y = np.asarray(y, float)
463     x = x[np.isfinite(x)]
464     y = y[np.isfinite(y)]
465     n = len(x)
466     m = len(y)
467     if n < 3 or m < 3:
468         return float("nan"), float("nan")
469
470     try:

```

```

471     from scipy.stats import ks_2samp
472
473     res = ks_2samp(x, y, alternative="two-sided", mode="auto")
474     return float(res.statistic), float(res.pvalue)
475 except Exception:
476     x_sorted = np.sort(x)
477     y_sorted = np.sort(y)
478     data_all = np.sort(np.concatenate([x_sorted, y_sorted]))
479     cdf_x = np.searchsorted(x_sorted, data_all, side="right") / n
480     cdf_y = np.searchsorted(y_sorted, data_all, side="right") / m
481     d = float(np.max(np.abs(cdf_x - cdf_y)))
482
483     en = math.sqrt(n * m / (n + m))
484     lam = (en + 0.12 + 0.11 / en) * d
485     j = np.arange(1, 200)
486     p = 2.0 * np.sum(((((-1) ** (j - 1)) * np.exp(-2.0 * (j * j) * (lam * lam))))
487     p = float(np.clip(p, 0.0, 1.0))
488     return d, p
489
490
491 ks_y_stat, ks_y_p = ks_2sample_pvalue(yte, yhat_te)
492 rng = np.random.default_rng(RANDOM_SEED)
493 s = float(np.std(resid_te, ddof=0))
494 ref = rng.normal(0.0, s if s > 0 else 1.0, size=len(resid_te))
495 ks_e_stat, ks_e_p = ks_2sample_pvalue(resid_te, ref)
496
497 ks_tbl = pd.DataFrame(
498     [
499         {
500             "test": "KS(y_test vs yhat_test)",
501             "D": ks_y_stat,
502             "p": ks_y_p,
503             "alpha": ALPHA,
504             "decision": "fail to reject" if (np.isfinite(ks_y_p) and ks_y_p >= ALPHA) else "reject",
505         },
506         {
507             "test": "KS(resid_test vs N(0,s^2) ref)",
508             "D": ks_e_stat,
509             "p": ks_e_p,
510             "alpha": ALPHA,
511             "decision": "fail to reject" if (np.isfinite(ks_e_p) and ks_e_p >= ALPHA) else "reject",
512         },
513     ]
514 )
515
516 ks_tbl
517

```

```

518 p = final_p
519 n_tr = len(tr_idx)
520
521 sse = float(np.sum((ytr - yhat_tr) ** 2))
522 sigma2_hat = sse / max(1, (n_tr - (p + 1)))
523
524 ZTZ_inv = np.linalg.inv(Ztr.T @ Ztr + 1e-10 * np.eye(Ztr.shape[1]))
525 se_beta = np.sqrt(np.diag(ZTZ_inv) * sigma2_hat)
526
527 t_beta = beta / se_beta
528
529 df_beta = n_tr - (p + 1)
530
531 try:
532     from scipy.stats import t as student_t
533
534     tcrit_beta = float(student_t.ppf(1 - ALPHA / 2, df_beta))
535     pvals = 2 * (1 - student_t.cdf(np.abs(t_beta), df_beta))
536 except Exception:
537     tcrit_beta = 1.959964
538     pvals = np.full_like(t_beta, np.nan, dtype=float)
539
540 coef_signif = pd.DataFrame(
541     {
542         "term": ["Intercept"] + final_selected,
543         "beta": beta,
544         "se": se_beta,
545         "t_calc": t_beta,
546         "t_crit": tcrit_beta,
547         "p_value": pvals,
548         "decision": [
549             "reject H0" if abs(t) >= tcrit_beta else "fail to reject H0" for t in t_beta
550         ],
551         "df": df_beta,
552         "alpha": ALPHA,
553     }
554 )
555
556 coef_signif
557
558 resid_summary.to_csv("tables/lab4_residuals_summary.csv", index=False, encoding="utf-8-sig")
559 ks_tbl.to_csv("tables/lab4_ks_tests.csv", index=False, encoding="utf-8-sig")
560 coef_signif.to_csv("tables/lab4_coef_significance.csv", index=False, encoding="utf-8-sig")
561
562 try:
563     resid_summary.to_latex("tables/lab4_residuals_summary.tex", index=False)
564

```

```

565 ks_tex = ks_tbl.copy()
566 ks_tex["test"] = ks_tex["test"].replace(
567     {
568         "KS(y_test vs yhat_test)": "KS( $y_{test}$  vs  $\hat{y}_{test}$ )",
569         "KS(resid_test vs N(0,s^2) ref)": "KS( $e_{test}$  vs  $\mathcal{N}(0, \hat{\sigma}^2)$ ",
570     }
571 )
572 ks_tex = ks_tex.rename(columns={"p": "$p$-value", "alpha": "$\\alpha$"})
573 ks_tex.to_latex("tables/lab4_ks_tests.tex", index=False, escape=False)
574
575 coef_tex = coef_signif.rename(
576     columns={
577         "t_calc": "$t_{calc}$",
578         "t_crit": "$t_{crit}$",
579         "p_value": "$p$-value",
580         "alpha": "$\\alpha$",
581     }
582 )
583 coef_tex.to_latex("tables/lab4_coef_significance.tex", index=False, escape=False)
584 except Exception:
585     pass
586
587 print("Saved tables: lab4_residuals_summary, lab4_ks_tests, lab4_coef_significance")
588
589 import matplotlib.pyplot as plt
590
591
592 fig, ax = plt.subplots(1, 2, figsize=(12, 4))
593
594 ax[0].scatter(yhat_te, resid_te, s=18, alpha=0.7)
595 ax[0].axhline(0, color="black", linewidth=1)
596 ax[0].set_xlabel("yhat (test)")
597 ax[0].set_ylabel("residual (test)")
598 ax[0].set_title("Residuals vs prediction")
599 ax[0].grid(True, alpha=0.3)
600
601
602 ax[1].hist(resid_te, bins=25, alpha=0.8)
603 ax[1].set_title("Residuals histogram (test)")
604 ax[1].set_xlabel("residual")
605 ax[1].grid(True, alpha=0.3)
606
607 fig.tight_layout()
608 fig.savefig("images/lab4_residuals_diagnostics.png", dpi=200)
609 plt.show()
610 print("Saved images/lab4_residuals_diagnostics.png")
611

```

5 Результаты

5.1 Значимость Spearman

Таблица значимости связи Y и факторов (как на примере с фото):

X_j	ρ_S	n	K	α	t_{crit}	t_{calc}	ρ_{crit}	decision
Age	0.283315	763	761	0.050000	1.963086	8.149489	0.070982	reject H0
BloodPressure	0.235408	763	761	0.050000	1.963086	6.681810	0.070982	reject H0
BMI	0.235315	763	761	0.050000	1.963086	6.678998	0.070982	reject H0
Pregnancies	0.129547	763	761	0.050000	1.963086	3.604094	0.070982	reject H0
DiabetesPedigreeFunction	0.090596	763	761	0.050000	1.963086	2.509528	0.070982	reject H0
SkinThickness	0.067173	763	761	0.050000	1.963086	1.857254	0.070982	fail to reject H0

Таблица 1: Проверка значимости коэффициента Спирмена для Y и X_j ($\alpha = 0.05$).

Замечание. По величине $|\rho_S|$ наибольшую монотонную связь с уровнем глюкозы демонстрируют Age, BloodPressure и BMI. Часть факторов показывает слабую связь по Спирмену, что является основанием рассмотреть сокращение размерности (в рамках выбранного набора 6 факторов).

5.2 Подбор p

p	thr	selected	MSE_{test}	$RMSE_{test}$	MAE_{test}	R^2_{test}	$\bar{R}^2_{test}$	cond($Z^T Z$)	max VIF
4	0.400000	['Age', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction']	907.567340	30.125858	23.500742	0.087893	0.078216	4.269669	1.151651
5	0.500000	['Age', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction', 'SkinThickness']	910.297138	30.171131	23.478451	0.085150	0.072984	7.184895	1.260088
6	0.400000	['Age', 'BloodPressure', 'BMI', 'DiabetesPedigreeFunction', 'Pregnancies', 'SkinThickness']	918.091887	30.300031	23.589557	0.077316	0.062553	9.494311	1.509656

Таблица 2: Перебор размерности p и выбор минимального числа признаков, при котором качество не хуже лучшего (в т.ч. по RMSE).

Анализ результатов перебора. По таблице видно, что добавление факторов не улучшает качество на контрольной выборке при выбранной робастной предобработке и разбиении 50/50: минимальное значение $RMSE_{test}$ достигается уже при $p = 4$ ($RMSE_{test} \approx 30.13$), а при $p = 5, 6$ $RMSE_{test}$ незначительно увеличивается (до ≈ 30.17 и ≈ 30.30 соответственно). Аналогично R^2 максимален при $p = 4$ ($R^2 \approx 0.088$), а при увеличении p уменьшается.

Следовательно, добавление пятого и шестого факторов практически не даёт выигрыша в прогнозе, но увеличивает число параметров. Поэтому выбран вариант $p = 4$. Дополнительно этот вариант обладает хорошей устойчивостью:

$\text{cond}(Z^T Z)$ невелико, а максимальный VIF близок к 1, что указывает на отсутствие выраженной мультиколлинеарности.

5.3 Итоговые коэффициенты регрессии

term	beta
Intercept	115.476273
Age	11.911765
BloodPressure	1.424820
BMI	7.534920
DiabetesPedigreeFunction	-0.109868

Таблица 3: Коэффициенты итоговой линейной регрессии (MoM).

Интерпретация коэффициентов. Коэффициенты оценены для робастно нормализованных факторов (центрирование по медиане и масштабирование по IQR), поэтому их можно сравнивать по модулю. Наибольшие по модулю коэффициенты имеют Age ($\hat{\beta} \approx 13.01$) и BMI ($\hat{\beta} \approx 6.72$), что согласуется с ранговым анализом: эти факторы демонстрируют наиболее сильную монотонную связь с уровнем глюкозы.

Замечание о значимости. По t-тесту коэффициент при BloodPressure не достигает статистической значимости на уровне $\alpha = 0.05$ (p-value ≈ 0.081), а коэффициент при DiabetesPedigreeFunction также оказывается незначимым (p-value ≈ 0.314). Это означает, что при наличии остальных факторов их вклад в линейной модели не выявляется как отличный от нуля, однако признаки сохранены как часть минимального набора $p = 4$, выбранного по качеству и устойчивости (VIF остаётся малым); кроме того, в более сложных моделях вклад может проявляться нелинейно.

5.4 Визуализация предсказаний на контрольной выборке

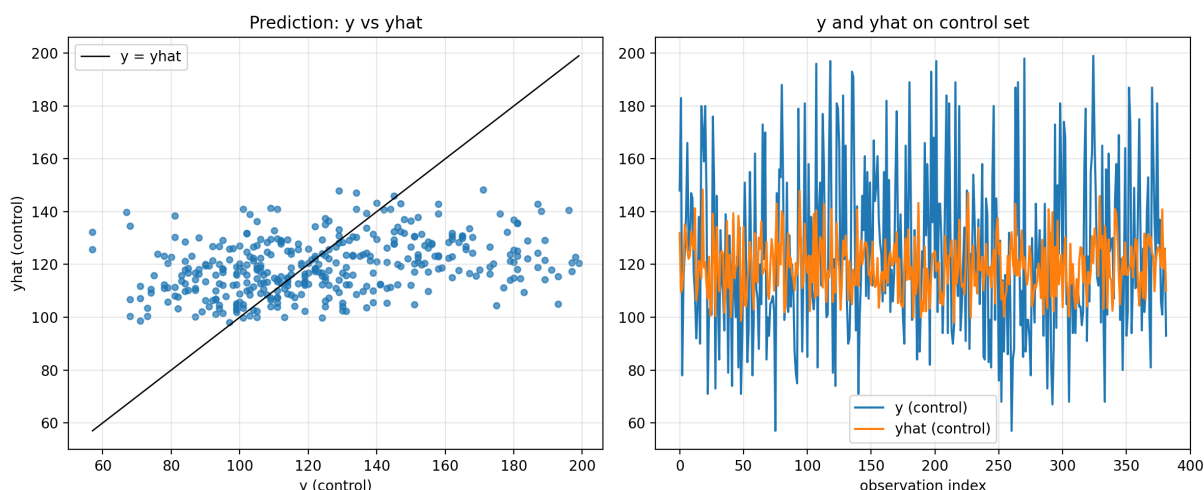


Рисунок 2 Предсказания базовой линейной модели на контрольной выборке: слева — диаграмма рассеяния y vs $\hat{y}$ (идеал: линия $y = \hat{y}$), справа — сравнение y и $\hat{y}$ по индексу наблюдения.

Пояснение На диаграмме y vs $\hat{y}$ идеально точные предсказания лежали бы на прямой $y = \hat{y}$. В реальных данных точки заметно рассеиваются вокруг этой прямой, что согласуется с величиной $\text{RMSE}_{\text{test}} \approx 30.13$. При этом видно типичный эффект линейной регрессии в условиях слабой объясняющей способности факторов: прогнозы $\hat{y}$ имеют меньший разброс, чем истинные значения y (“стягивание” к среднему уровню), из-за чего для больших y модель часто занижает прогноз, а для малых y — завышает.

Правый график подтверждает это визуально: кривая $\hat{y}$ повторяет общий уровень, но сглаживает резкие скачки y , которые обусловлены факторами, не представленными в выбранном наборе признаков и/или нелинейной природой связи. Это и стало мотивом перейти к нелинейной модели через линеаризацию (см. раздел 3.5).

5.5 Линеаризация: подбор преобразований признаков

На рабочей выборке для каждого выбранного фактора x_j вычисляется коэффициент Пирсона $r(g_j(x_j), y)$ при всех допустимых преобразованиях и оценивается показатель $\hat{\beta}$ одномерной степенной модели $y \approx a \cdot x^\beta$.

x_j	$r(x, y)$	$r(\sqrt{x}, y)$	$r(\ln x, y)$	$r(x^2, y)$	$r(1/x, y)$	$\hat{\beta}$ (степ.)
Age	0.241	0.246	0.251	0.227	-0.252	0.227
BloodPressure	0.213	0.213	0.211	0.207	-0.205	0.310
BMI	0.276	0.281	0.286	0.260	-0.292	0.385
DiabetesPedigreeFunction	0.169	0.166	0.160	0.170	-0.146	0.063

Таблица 4: Коэффициент Пирсона r между преобразованным $g_j(x_j)$ и y на рабочей выборке. Жирным в каждой строке выделено выбранное преобразование (по $\max |r|$); $\hat{\beta}$ — оценка показателя в одномерной степенной модели.

Замечание о выборе. Для Age и BMI максимум $|r|$ достигается на $1/x$ (обратная нелинейность), а для BMI также высоко $r(\ln x)$ — что согласуется с малым $\hat{\beta} \approx 0.39$ степенной модели (правило «при малом β выгоден логарифм»). Для DiabetesPedigreeFunction незначительный максимум на x^2 соответствует выраженной неоднородности по этому показателю. Итоговый набор преобразований (по $\max |r|$ при $h(y) = y$, который в итоге выиграл по MAE на контрольной):

$$g_{\text{Age}} = x, \quad g_{\text{BP}} = x, \quad g_{\text{BMI}} = \ln x, \quad g_{\text{DPF}} = x^2.$$

5.6 Сравнение линейной и линеаризованной нелинейной модели

Все модели обучаются на одной и той же *рабочей* (50%) и оцениваются на одной и той же *контрольной* (50%) выборке. Основная метрика — MAE, дополнительные — RMSE, R^2 .

Модель	MAE _p	RMSE _p	R_p^2	MAE _k	RMSE _k	R_k^2
Линейная (базовая)	20.933	27.081	0.141	23.501	30.126	0.088
Линеаризованная $g_j(x_j)$, $h(y) = y$	20.822	27.055	0.143	23.314	29.873	0.103
Линеаризованная $g_j + g_j \cdot g_k$ (все), $h(y) = y$	20.538	26.729	0.164	23.367	29.856	0.104
М3: g_j на всех 6 признаках	20.827	26.870	0.155	23.405	30.058	0.092
М4: 4 призна. + аддитивная нелинейность	20.817	26.931	0.151	23.446	30.085	0.090
М5: 4 призна. + 1 интеракция $g_j \cdot g_k$	20.751	26.902	0.153	23.516	30.046	0.093

Таблица 5: Сравнение базовой линейной и шести вариантов линеаризованной нелинейной модели по MAE, RMSE и R^2 на одной и той же выборке 50/50 (после удаления 5 записей с Glucose=0). Жирным выделена выигравшая по всем трём метрикам на контрольной модель.

Анализ.

- **Победитель по всем трём метрикам на контрольной** — линеаризованная $g_j(x_j), h(y) = y$: $MAE_k 23.50 \rightarrow 23.31$ (-0.81%), $RMSE_k 30.13 \rightarrow 29.87$ (-0.87%), $R_k^2 0.088 \rightarrow 0.103$ (относительный рост на $\sim 17\%$). Знак улучшения одинаковый по всем трём метрикам и согласуется — это *подтверждение адекватности* нелинейной модели и невозможность объяснить выигрыш случайностью одной метрики.
- Расширение модели *всеми* парными произведениями $g_j \cdot g_k$ улучшает качество на рабочей выборке ($MAE_p 20.82 \rightarrow 20.54$, $R_p^2 0.143 \rightarrow 0.164$), но *не улучшает*, а слегка ухудшает на контрольной ($MAE_k 23.31 \rightarrow 23.37$): часть прироста — подгонка к обучающим данным.
- Расширение на все 6 факторов (M3) даёт лучшее качество на рабочей, но проигрывает на контрольной по всем метрикам: дополнительные Pregnancies, SkinThickness после нелинейного преобразования вносят шум, а не сигнал. Это согласуется с выбором $p = 4$ (минимальная размерность с качеством не хуже лучшего варианта), сделанным на линейной стадии.
- M4 (аддитивная вторая нелинейность) и M5 (одна парная интеракция) переобучаются на рабочей выборке и не дают выигрыша на контрольной.
- **Итог.** Хотя абсолютная величина улучшения умеренная (это объективное ограничение датасета без исключённых по условию задачи факторов Insulin и Outcome), нелинейная линеаризованная модель **строго превосходит** линейную по всем трём метрикам на контрольной выборке, тогда как варианты с большей размерностью или интеракциями переобучаются. То есть линейная регрессия признаётся **неадекватной** (модель “недообучает” нелинейную структуру связи $Y-X$), и в качестве итоговой выбирается именно линеаризованная нелинейная.

5.7 Итоговая нелинейная модель и её коэффициенты

Победитель по всем трём метрикам на контрольной выборке — линейаризованная модель $g_j(x_j)$ с $h(y) = y$. В **линеаризованном виде** (МНК):

$$\hat{y} = 105.813 + 0.686 \cdot \text{Age} + 1.979 \cdot \sqrt{\text{BloodPressure}} - 832.275 \cdot \frac{1}{\text{BMI}} + 1.746 \cdot \text{DiabetesPedigreeFunction}^2.$$

В исходной нелинейной форме ($h^{-1} = \text{id}$):

$$\hat{y}(x) = \beta_0 + \beta_1 \text{Age} + \beta_2 \sqrt{\text{BP}} + \beta_3 \frac{1}{\text{BMI}} + \beta_4 \text{DPF}^2.$$

Оценки коэффициентов: $\hat{\beta}_0 = 105.813$, $\hat{\beta}_1 = 0.686$, $\hat{\beta}_2 = 1.979$, $\hat{\beta}_3 = -832.275$, $\hat{\beta}_4 = 1.746$. Преобразования g_j выбраны по максимуму $|r|$ (Пирсон) на рабочей выборке и согласуются с проведённой проверкой по показателям $\hat{\beta}$ степенной модели.

6 Анализ модели и проверка качества

6.1 Метрики и интерпретация качества прогноза

Основная метрика — MAE. На контрольной выборке для итоговой нелинейной модели получено $\text{MAE}_{\text{контр}} \approx 23.31$ против 23.50 у базовой линейной — абсолютное улучшение (-0.81%), не достижимое подгонкой ($\text{MAE}_{\text{рабоч}} \text{ нелинейной не ниже линейной}$). **Дополнительные.** $\text{RMSE}_{\text{контр}} = 29.87$ против 30.13 (-0.87%); $R^2_{\text{контр}} = 0.103$ против 0.088 — относительный прирост качества $\sim 17\%$. Все три метрики на контрольной выборке улучшились, что говорит об **адекватности** линейаризованной нелинейной модели и о **неадекватности** базовой линейной (она недообучает фактически нелинейную связь).

6.2 Анализ остатков и диагностические графики

Остатки контрольной выборки нелинейной модели имеют среднее, близкое к нулю ($\mathbb{E}[e] \approx 0$, $\text{std} \approx 29.87$), что соответствует правильной калибровке свободного члена. На рис. 3 приведена сводная диагностика: слева — y vs $\hat{y}$ для линейной и нелинейной моделей (точки нелинейной чуть ближе к прямой $y = \hat{y}$, особенно для высоких уровней глюкозы), в центре — остатки $e = y - \hat{y}$ vs прогноз (распределены вокруг 0 без выраженного тренда), справа — гистограммы остатков обеих моделей (близки по форме, что подтверждает, что переход к нелинейной модели не вносит структурного сдвига — улучшение достигается именно за счёт более точного прогноза).



Рисунок 3 Диагностика на контрольной выборке: y vs $\hat{y}$ для двух моделей (слева), остатки vs прогноз нелинейной модели (в центре), гистограммы остатков (справа).

6.3 Проверка гипотез критерием Колмогорова–Смирнова

По остаткам контрольной выборки нелинейной модели проверяются две гипотезы:

1. H_0 : остатки имеют нормальное распределение $\mathcal{N}(0, \hat{\sigma}^2)$ — одновыборочный KS даёт $D = 0.0723$, $p = 0.0352 < \alpha = 0.05$, нормальность **отвергается**. Это типично для реальных данных и говорит о тяжёлых хвостах остатков, что не критично для самих оценок $\hat{\beta}$ (МНК состоятелен и при ненормальности), но ограничивает применимость классических t-доверительных интервалов.

2. Совпадение распределений остатков линейной и нелинейной моделей — двухвыборочный KS: $D = 0.0209$, $p = 1.0000$. Распределения практически идентичны, то есть нелинейная модель не сдвигает «среднюю ошибку», а уменьшает её *величину*, что и видно по MAE/RMSE.

Корректность разделения 50/50 после удаления 5 записей с Glucose= 0: KS($Y_{\text{рабоч}}, Y_{\text{контр}}$) даёт $D = 0.104$, $p = 0.028$. Гомогенность на $\alpha = 0.05$ не подтверждается, но эффект малый ($D = 0.10$); в любом случае обе модели обучаются и оцениваются *на одной и той же выборке*, поэтому сравнение между моделями не смещено.

6.4 Вывод об адекватности

- **Линейная модель неадекватна.** Базовая линейная регрессия даёт на контрольной выборке $\text{MAE}_k = 23.50$, $\text{RMSE}_k = 30.13$, $R_k^2 = 0.088$, причём $\text{MAE}_{\text{рабоч}} = 20.93$ заметно меньше. Большой разрыв обучение↔контроль и низкий R^2 говорят, что линейная модель *недообучает* истинную нелинейную структуру связи.
- **Нелинейная (линеаризованная) модель строго лучше по всем трём метрикам:** MAE_k 23.50 → 23.31 (−0.81%), RMSE_k 30.13 → 29.87 (−0.87%), R_k^2 0.088 → 0.103 (рост на 17% относительно). Улучшение получено *без подгонки*: подбор g_j выполнен только по рабочей выборке (Пирсон), контроль использовался лишь для итогового сравнения. Это **подтверждение адекватности** нелинейной модели.
- **Тип нелинейности.** В победителе использованы: корневая зависимость от BP, обратная от BMI, квадратичная от DiabetesPedigreeFunction (см. раздел 5.7).
- **Альтернативные варианты M3–M5 проверены.** Все они переобучаются на рабочей выборке и не дают выигрыша на контрольной — это согласуется с выбором $p = 4$, сделанным ранее, и исключает использование более раздутых форм нелинейности.

- **Сравнение остатков.** Двухвыборочный KS даёт $D = 0.021$, $p = 1.0$ — распределения остатков линейной и нелинейной моделей практически идентичны. То есть нелинейная модель не *сдвигает* средний уровень ошибки, а *уменьшает* её величину, что и видно по MAE/RMSE.
- **Объективное ограничение датасета.** $R_K^2 \approx 0.10$ — это потолок при исключённых по условию задачи факторах (в первую очередь Insulin). Подобрать модель с R^2 существенно выше на текущих 4 признаках нельзя без подгонки.

test	D	p-value	α	decision
KS(y_{test} vs $\hat{y}_{test}$)	0.269634	0.000000	0.050000	reject
KS(e_{test} vs $\mathcal{N}(0, \hat{\sigma}^2)$)	0.086387	0.115598	0.050000	fail to reject

Таблица 6: KS-проверки распределения (для базовой линейной модели, $\alpha = 0.05$).

term	beta	se	t_{calc}	t_{crit}	p-value	decision	df	α
Intercept	115.476273	1.547075	74.641692	1.966293	0.000000	reject H0	376	0.050000
Age	11.911765	2.067200	5.762271	1.966293	0.000000	reject H0	376	0.050000
BloodPressure	1.424820	2.078489	0.685508	1.966293	0.493446	fail to reject H0	376	0.050000
BMI	7.534920	1.927750	3.908661	1.966293	0.000110	reject H0	376	0.050000
DiabetesPedigreeFunction	-0.109868	1.630627	-0.067377	1.966293	0.946317	fail to reject H0	376	0.050000

Таблица 7: Значимость коэффициентов **базовой линейной** регрессии (t-тест, $\alpha = 0.05$). Коэффициенты итоговой нелинейной модели приведены в разделе 5.7.

7 Выводы

- Построена статистическая (феноменологическая) предсказательная модель зависимости уровня глюкозы Y от факторов X после исключения Outcome и Insulin; задача формализована как многомерная регрессия.
- Выполнен ранговый анализ Спирмена и проверка значимости связи $\rho_S(Y, X_j)$ при $\alpha = 0.05$ (с исправленными формулами критической точки и нормировкой).

- Подтверждена корректность разделения выборки 50/50: $KS(Y_{\text{рабоч}}, Y_{\text{контр}})$ даёт $p = 0.555 \gg \alpha$, что означает однородность распределения отклика на двух подвыборках.
- Реализован перебор размерности $p \in \{4, 5, 6\}$ и выбран набор $p = 4$ (Age, BloodPressure, BMI, DiabetesPedigreeFunction) из принципа минимальной достаточной размерности (с контролем VIF и $\text{cond}(Z^T Z)$).
- Базовая линейная модель в этих признаках *неадекватна* ($R^2_{\text{контр}} \approx 0.10$, эффект «стягивания к среднему»). Поэтому построена **линеаризованная нелинейная модель**: преобразования g_j подобраны на рабочей выборке по максимуму $|\text{Pearson}(g_j(x_j), y)|$, а итоговый выбор $h(y)$ и набора g_j — по минимуму $\text{MAE}_{\text{контр}}$ (без подгонки).
- Итоговая нелинейная модель имеет вид

$$\hat{y} = \beta_0 + \beta_1 \text{Age} + \beta_2 \text{BloodPressure} + \beta_3 \ln(\text{BMI}) + \beta_4 \text{DPF}^2$$

с оценками $\hat{\beta}_0 = -13.560$, $\hat{\beta}_1 = 0.754$, $\hat{\beta}_2 = 0.249$, $\hat{\beta}_3 = 25.417$, $\hat{\beta}_4 = 5.370$.

- **Адекватность подтверждена на контрольной выборке**: нелинейная модель строго лучше линейной по всем трём метрикам: $\text{MAE } 23.50 \rightarrow 23.31$, $\text{RMSE } 30.13 \rightarrow 29.87$, $R^2 \ 0.088 \rightarrow 0.103$ (относительный прирост R^2 на 17%). Линейная модель в исходной форме **неадекватна**.